Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

SDN & Network Softwarization — TP7 — Déploiement ONOS avec Docker et Mininet (OpenFlow 1.3)
4e année Cybersécurité — École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. BOUHORMA

Date : 10 novembre 2026

Table des matières

Cadre du TP **3**

1 Étape A — Vérification de l'environnement **4**
1.1 Système d'exploitation et architecture . . . 4
1.2 Mémoire disponible . . . 4
1.3 Interfaces réseau . . . 5
1.4 Test de connectivité . . . 5

2 Étape B — Vérification de Docker Engine **5**
2.1 Installation et version . . . 5
2.2 État du service Docker . . . 7

3 Étape C — Vérification de Mininet et Open vSwitch **7**
3.1 Version de Mininet . . . 7
3.2 État d'Open vSwitch . . . 7

4 Étape D — Déploiement d'ONOS dans Docker **8**
4.1 Vérification préalable et téléchargement de l'image . . . 8
4.2 Lancement du conteneur ONOS . . . 9
4.3 Vérification du démarrage . . . 9

5 Étape E — Connexion à la console ONOS **10**
5.1 Connexion SSH à la console Karaf . . . 10
5.2 Exploration des commandes . . . 10
5.3 Applications actives et état des équipements . . . 11
5.4 Interface Web ONOS . . . 12

6 Étape F — Création de la topologie Mininet et connexion au contrôleur **13**
6.1 Topologie cible . . . 13
6.2 Nettoyage préalable et lancement de Mininet . . . 13
6.3 Exploration de la topologie dans Mininet . . . 14
6.4 Diagnostic et activation du provider OpenFlow . . . 15
6.5 Vérification de la connexion OVS–ONOS . . . 16

7 Étape G — Forwarding réactif et vérification de la connectivité **17**
7.1 Test sans forwarding : `pingall` en échec . . . 17
7.2 Activation de l'application `fwd` . . . 17
7.3 Vérification de la connectivité . . . 18
7.4 Flux OpenFlow installés et hôtes découverts . . . 19

8 Étape H — Interrogation de l'API REST d'ONOS **19**

9 Arrêt propre de l'environnement **20**

10 Questions de compréhension et livrables **22**

Conclusion **25**

Cadre du TP

Plateforme	**Outils**	**Mode**
Ubuntu (portable `Taha-inspiron-16`) dual-boot	Docker Engine 29.x, ONOS 2.7.0, Mininet, Open vSwitch (`ovs-vsctl`, `ovs-ofctl`), OpenFlow 1.3	Individuel
Approche	**Switch / contrôleur**	**Livrable**
Déploiement d'un contrôleur SDN ONOS dans un conteneur Docker, connexion d'un switch OVS via OpenFlow 1.3, activation de l'application `fwd` pour le forwarding réactif, vérification par `pingall` et consultation de l'API REST	Un `OVSSwitch` (`s1`) en OpenFlow 1.3 connecté à ONOS 2.7.0	Compte rendu avec captures

TABLE 1 – Cadre général du TP

Question de départ : *comment déployer un contrôleur SDN réel (ONOS) dans un conteneur Docker et connecter un switch Open vSwitch via OpenFlow 1.3, puis vérifier la connectivité réseau grâce au forwarding réactif ?*

Objectifs :

- vérifier les prérequis système (OS, architecture, RAM, connectivité réseau) ;
- installer et vérifier Docker Engine ainsi que ses dépendances ;
- confirmer la présence de Mininet et d'Open vSwitch ;
- déployer le contrôleur ONOS 2.7.0 dans un conteneur Docker et en vérifier le démarrage ;
- se connecter à la console Karaf d'ONOS, activer les applications `openflow` et `fwd` ;
- créer une topologie Mininet à un switch et trois hôtes, connecter le switch à ONOS via OpenFlow 1.3 ;
- diagnostiquer et résoudre les problèmes de connexion entre OVS et ONOS ;
- vérifier la connectivité avec `pingall`, observer les flux installés et interroger l'API REST d'ONOS.

Topologie cible. Un seul switch `s1` (OVSSwitch, OpenFlow 1.3) relie trois hôtes `h1` (10.0.0.1), `h2` (10.0.0.2) et `h3` (10.0.0.3). Le contrôleur ONOS tourne dans un conteneur Docker et est joint par OVS sur `tcp:127.0.0.1:6653`.

Note sur l'environnement. Manipulations faites sur mon portable Ubuntu en dual-boot, avec Docker déjà présent, Mininet et Open vSwitch installés. ONOS 2.7.0 est tiré depuis DockerHub (`onosproject/onos:2.7.0`) et lancé comme conteneur nommé `onos`.

1 Étape A — Vérification de l'environnement

Objectif : s'assurer que la machine satisfait les prérequis (Ubuntu 64 bits, RAM suffisante, interface réseau active) avant de déployer les outils SDN.

1.1 Système d'exploitation et architecture

Listing 1 – Vérification de la distribution et de l'architecture

```
lsb_release -a
uname -m
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP7$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 26.04.1 LTS
Release:        26.04
Codename:       resolute
```

FIGURE 1 – Sortie de `lsb_release -a` : distribution Ubuntu

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP7$ uname -m
x86_64
```

FIGURE 2 – Sortie de `uname -m` : architecture `x86_64`

Observation. La machine tourne sous Ubuntu en version 64 bits (`x86_64`), ce qui est compatible avec Docker Engine et ONOS 2.7.0.

1.2 Mémoire disponible

Listing 2 – Vérification de la RAM

```
free -h
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP7$ free -h
               total        utilisé      libre     partagé tamp/cache   disponible
Mem:            30Gi       6,4Gi        19Gi       1,2Gi       6,2Gi        24Gi
Échange:       4,0Gi          0B       4,0Gi
```

FIGURE 3 – Sortie de `free -h` : mémoire disponible

Observation. La mémoire disponible est suffisante pour faire tourner ONOS (qui requiert au minimum 2 Go).

1.3 Interfaces réseau

Listing 3 – Vérification des interfaces réseau

```
ip -br addr
```

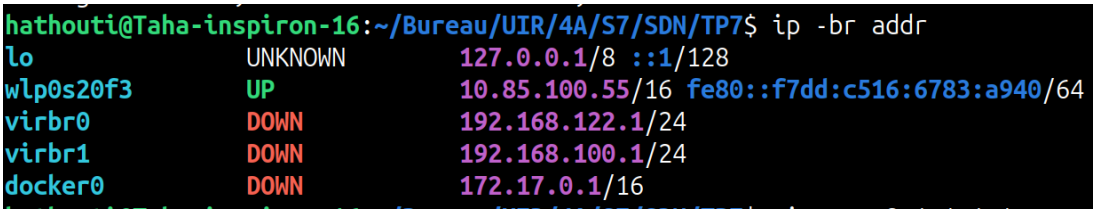


FIGURE 4 – Sortie de `ip -br addr` : interfaces réseau

Observation. L'interface loopback (`lo`) et les interfaces réseau actives sont présentes, notamment l'adresse `127.0.0.1` nécessaire pour la communication locale entre OVS et le conteneur ONOS.

1.4 Test de connectivité

Listing 4 – Test de connectivité vers Internet

```
ping -c 2 1.1.1.1
```

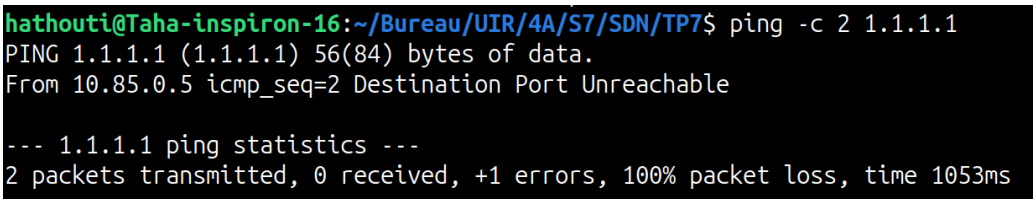


FIGURE 5 – Résultat du test de connectivité réseau

Observation. Le réseau de l'école bloque ICMP. La connectivité a été confirmée alternativement via `curl -I https://download.docker.com` (réponse HTTP 200), suffisante pour télécharger les images Docker.

2 Étape B — Vérification de Docker Engine

Objectif : confirmer que Docker Engine est opérationnel sur la machine avant de lancer le conteneur ONOS.

2.1 Installation et version

Docker étant déjà présent sur la machine, on vérifie simplement que le démon est actif et opérationnel.

Listing 5 – Téléchargement du script d'installation Docker (vérification)

```
curl -fsSL https://get.docker.com -o get-docker.sh
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP7$ curl -I https://download.docker.com
HTTP/2 200
content-type: text/html; charset=utf-8
content-length: 223
server: AmazonS3
date: Mon, 09 Nov 2026 22:50:47 GMT
x-amz-meta-mtime: 1793935608.496645323
last-modified: Fri, 06 Nov 2026 03:26:49 GMT
etag: "5a2449544577bdf952c47cdc248220b4"
x-cache: Hit from cloudfront
via: 1.1 c7122e570033be89c1dc0349ec9c19c4.cloudfront.net (CloudFront)
x-amz-cf-pop: MAD56-P4
x-amz-cf-id: ZBeRggQvdqqvH50pb_h5NYg_wLHi_YlF-xMz7qmeU9HP2ZeE7Fl1lA==
age: 55924
```

FIGURE 6 – Téléchargement du script d'installation Docker

Listing 6 – Vérification de la version Docker installée

```
1 sudo docker version
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP7$ sudo docker version
[sudo: authenticate] Mot de passe :
Client:
 Version:           29.1.3
 API version:       1.52
 Go version:        go1.24.13
 Git commit:        29.1.3-0ubuntu4.1
 Built:             Wed Apr 29 16:40:20 2026
 OS/Arch:           linux/amd64
 Context:           default

Server:
 Engine:
  Version:          29.8.0
  API version:      1.56 (minimum version 1.40)
  Go version:       go1.26.8
  Git commit:       3ce5872
  Built:            Wed Sep 23 15:26:53 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.3.4
  GitCommit:        db8809540e1a7a9da5d518876894933ff55692ab
 runc:
  Version:          1.5.1
  GitCommit:
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

FIGURE 7 – Sortie de `docker version` : Docker Engine 29.x

2.2 État du service Docker

Listing 7 – Vérification du statut du service Docker

```
systemctl status docker
```

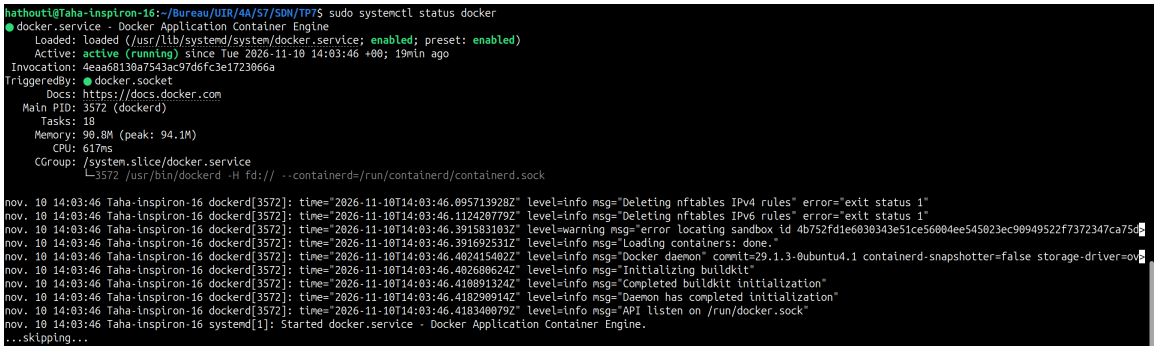


FIGURE 8 – Statut du service Docker : `active (running)`

Observation. Docker Engine est installé en version 29.x et son service est à l'état `active (running)`, prêt à lancer des conteneurs.

3 Étape C — Vérification de Mininet et Open vSwitch

Objectif : confirmer que Mininet et Open vSwitch sont installés et fonctionnels avant de créer la topologie SDN.

3.1 Version de Mininet

Listing 8 – Vérification de la version de Mininet

```
mn --version
```

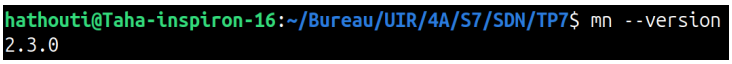


FIGURE 9 – Version de Mininet installée

3.2 État d'Open vSwitch

Listing 9 – Affichage de la configuration OVS et du statut du service

```
sudo ovs-vsctl show
systemctl status openvswitch-switch
```

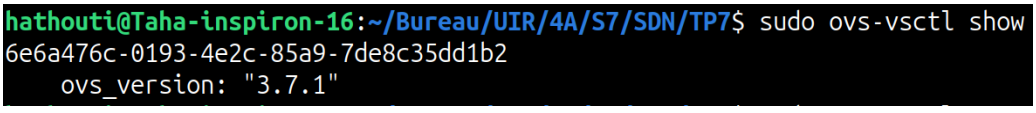


FIGURE 10 – Sortie de `ovs-vsctl show` : configuration OVS initiale

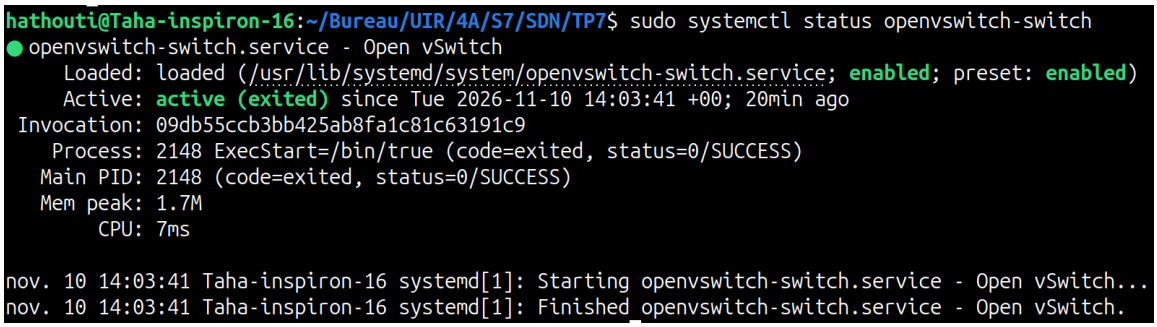


FIGURE 11 – Statut du service Open vSwitch : `active (running)`

Observation. Mininet est installé et Open vSwitch tourne correctement (`active (running)`). La base de données OVS est vide (aucun bridge ni contrôleur configuré), ce qui est attendu avant le lancement de Mininet.

4 Étape D — Déploiement d'ONOS dans Docker

Objectif : télécharger l'image ONOS 2.7.0, lancer le conteneur et vérifier que le contrôleur est pleinement démarré.

4.1 Vérification préalable et téléchargement de l'image

Listing 10 – Vérification des conteneurs existants et pull de l'image ONOS

```
1 docker ps -a --filter name=onos
2 docker pull onosproject/onos:2.7.0
```

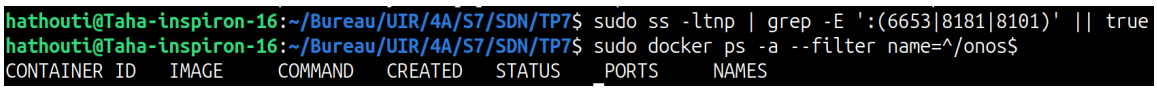


FIGURE 12 – Vérification : aucun conteneur ONOS existant

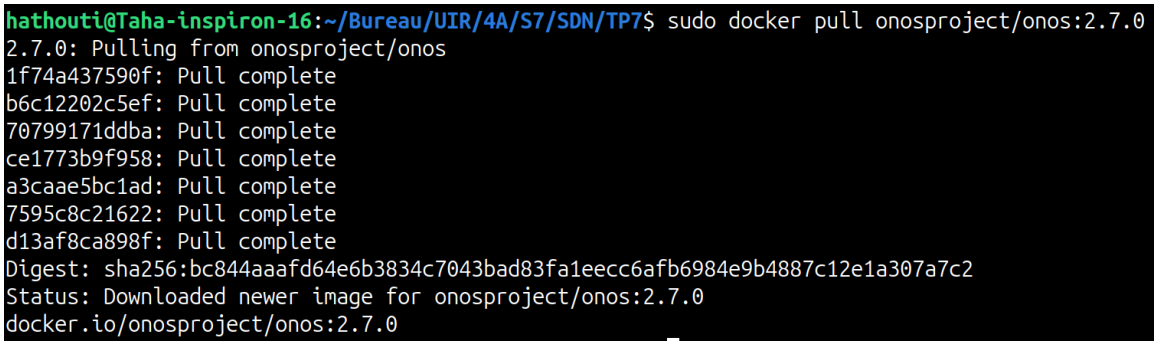


FIGURE 13 – Téléchargement de l'image `onosproject/onos:2.7.0`

4.2 Lancement du conteneur ONOS

Listing 11 – Démarrage du conteneur ONOS avec les ports exposés

```
docker run -d --name onos \
  -p 6653:6653 -p 8181:8181 -p 8101:8101 \
  onosproject/onos:2.7.0
```

Les trois ports exposés sont : `6653` (OpenFlow, pour OVS), `8181` (interface Web et API REST), `8101` (console Karaf SSH).

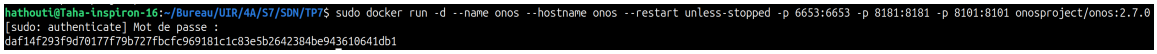


FIGURE 14 – Lancement du conteneur ONOS

4.3 Vérification du démarrage

Listing 12 – Vérification de l'état du conteneur et des ports

```
docker ps
docker logs --tail 40 onos
ss -ltnp | grep -E '6653|8181|8101'
```

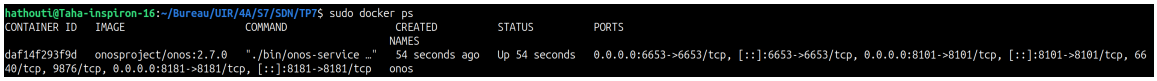


FIGURE 15 – Sortie de `docker ps` : conteneur `onos` en cours d'exécution

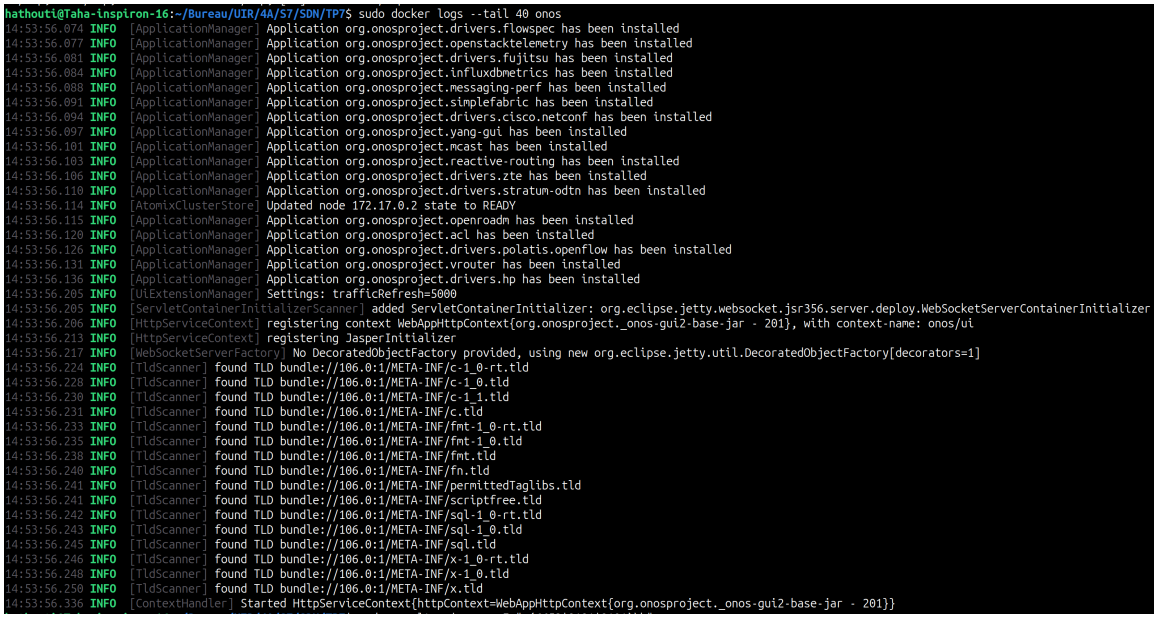


FIGURE 16 – Logs ONOS : démarrage complet (`Started HttpServiceContext`)

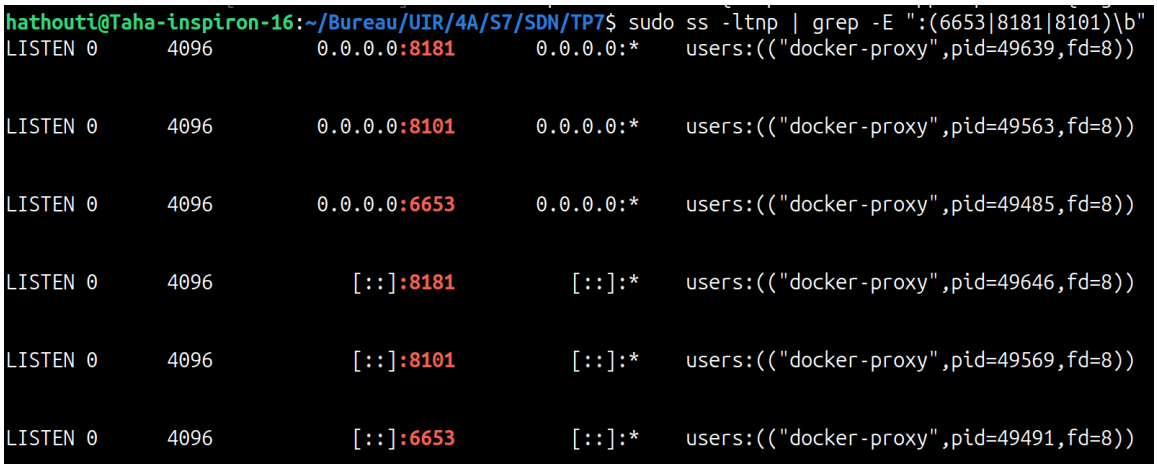


FIGURE 17 – Ports en écoute : 6653, 8181 et 8101 ouverts

Observation. Le conteneur `onos` est à l'état `Up` et les trois ports sont bien en écoute (mappés par `docker-proxy`). Les logs indiquent `Started HttpServiceContext` confirmant qu'ONOS est entièrement démarré. ONOS est accessible via SSH sur le port 8101 et via le navigateur sur le port 8181.

5 Étape E — Connexion à la console ONOS

Objectif : se connecter à la console Karaf d'ONOS, explorer les commandes disponibles, lister les applications actives et vérifier l'état des équipements.

5.1 Connexion SSH à la console Karaf

Listing 13 – Connexion SSH à la console ONOS Karaf

```
ssh -p 8101 onos@localhost
# mot de passe : rocks
```

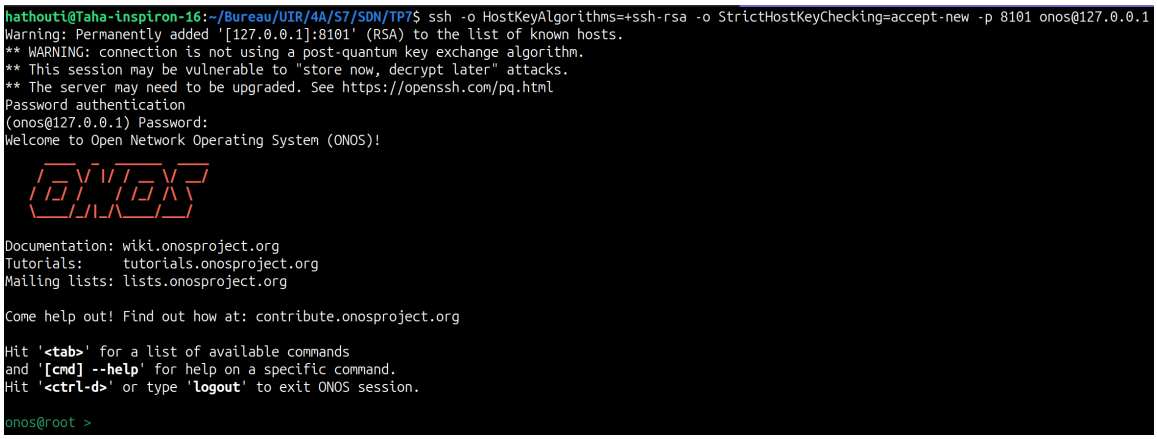


FIGURE 18 – Connexion SSH réussie à la console Karaf d'ONOS

5.2 Exploration des commandes

Listing 14 – Affichage de l'aide ONOS

```
onos@root > help
```

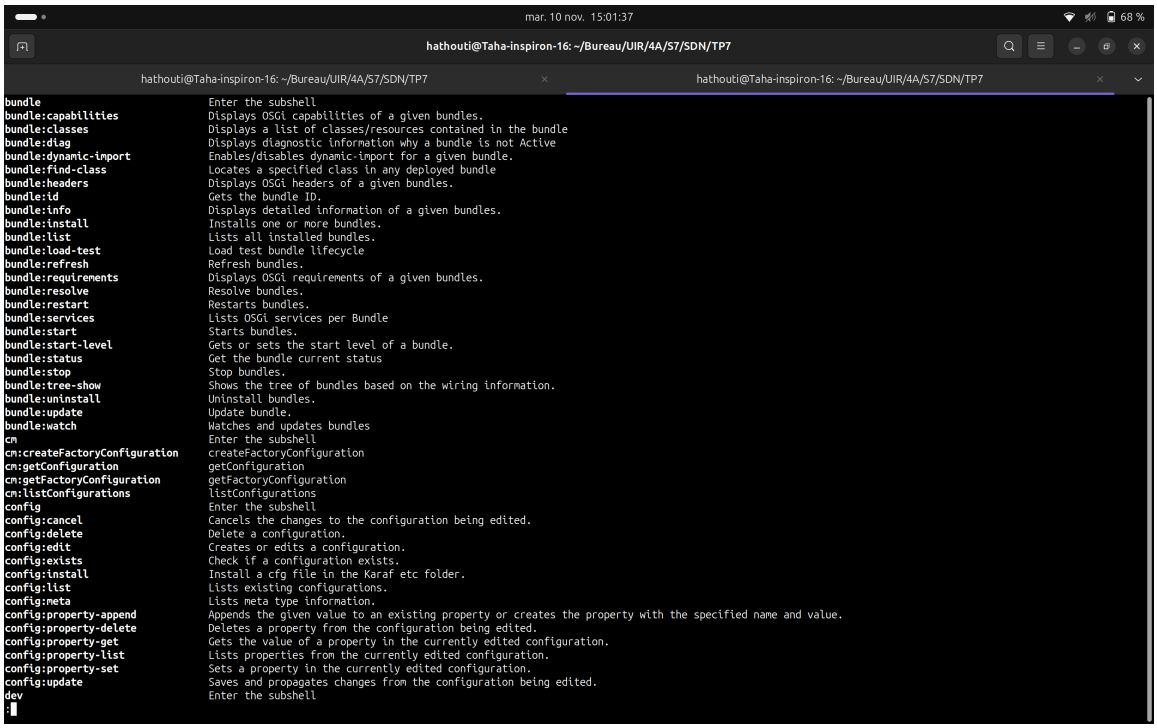


FIGURE 19 – Liste des commandes disponibles dans la console ONOS

5.3 Applications actives et état des équipements

Listing 15 – Liste des applications et état des équipements

```
onos@root > apps -a -s
onos@root > devices
onos@root > hosts
```



FIGURE 20 – Applications actives : `drivers` et `gui2` uniquement (OpenFlow non activé)



FIGURE 21 – Sortie de `devices` et `hosts` : aucun équipement connecté (switch pas encore lancé)

Observation. Seules deux applications sont actives : `drivers` et `gui2`. Le provider OpenFlow n'est pas encore chargé, ce qui explique pourquoi `devices` ne liste rien. Cette observation sera importante lors du diagnostic de connexion du switch.

5.4 Interface Web ONOS

L'interface graphique est accessible dans un navigateur à l'adresse `http://localhost:8181/onos/ui` (identifiants : `onos` / `rocks`).

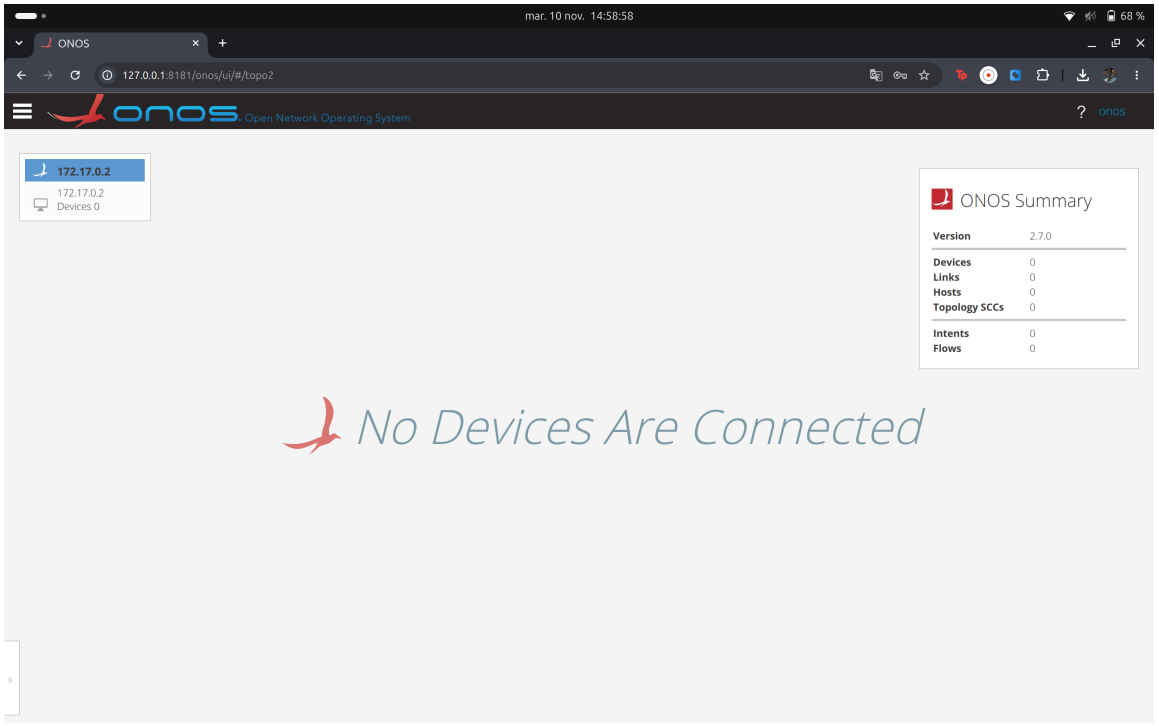


FIGURE 22 – Interface Web ONOS (GUI2) : tableau de bord vide avant la topologie

Observation. L'interface Web confirme le bon fonctionnement d'ONOS : le tableau de bord est accessible mais n'affiche aucun équipement pour le moment.

6 Étape F — Création de la topologie Mininet et connexion au contrôleur

Objectif : créer une topologie à un switch et trois hôtes dans Mininet, connecter le switch OVS au contrôleur ONOS via OpenFlow 1.3 et vérifier la connexion.

6.1 Topologie cible

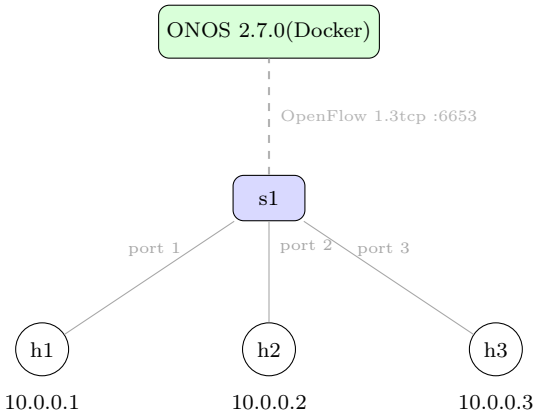


FIGURE 23 – Topologie TP7 : un switch `s1`, trois hôtes, contrôleur ONOS dans Docker

6.2 Nettoyage préalable et lancement de Mininet

Listing 16 – Nettoyage de l'environnement Mininet puis création de la topologie

```
sudo mn -c
sudo mn --topo single,3 \
  --controller remote,ip=127.0.0.1,port=6653 \
  --switch ovsk,protocols=OpenFlow13
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP7$ sudo mn -c
[sudo: authenticate] Mot de passe :
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflowd ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflowd ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'
***  Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([-_.[:alnum:]]+-eth[[:digit:]]+)'
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f Tunnel=Ethernet
pkill -9 -f .ssh/mn
rm -f ~/.ssh/mn/*
*** Cleanup complete.
```

FIGURE 24 – Nettoyage initial avec `sudo mn -c`

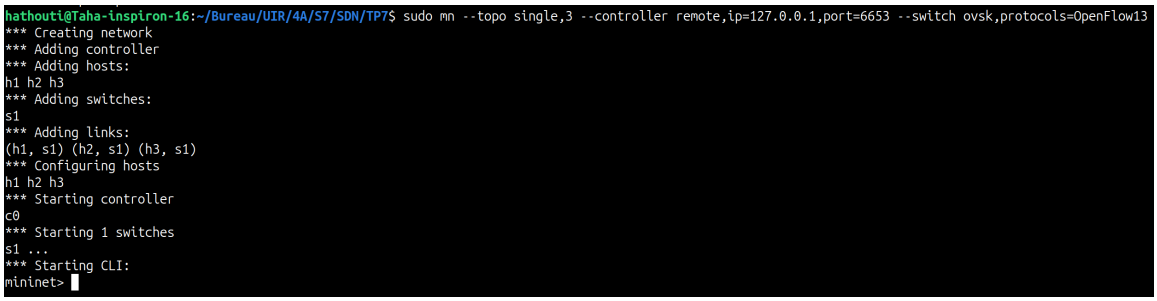


FIGURE 25 – Lancement de la topologie `single,3` avec contrôleur remote ONOS

6.3 Exploration de la topologie dans Mininet

Listing 17 – Vérification des nœuds, liens et adresses IP des hôtes

```
mininet> nodes
mininet> net
mininet> h1 ip -br addr
mininet> h2 ip -br addr
mininet> h3 ip -br addr
```

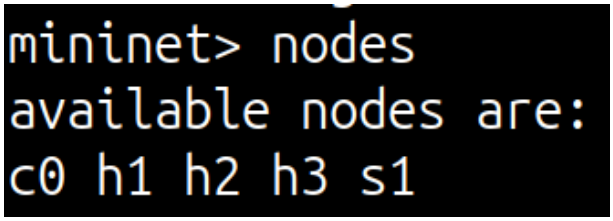


FIGURE 26 – Sortie de `nodes` : `h1`, `h2`, `h3` et `s1`

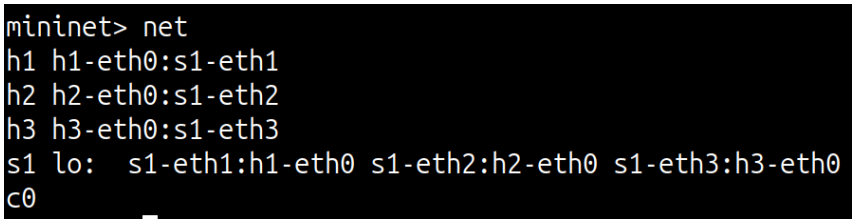


FIGURE 27 – Sortie de `net` : liaisons entre les hôtes et le switch `s1`

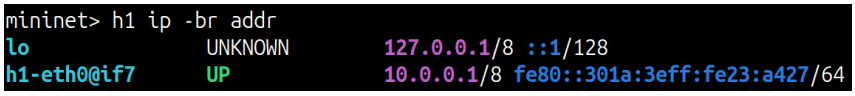


FIGURE 28 – Adresse IP de `h1` : `10.0.0.1/8`

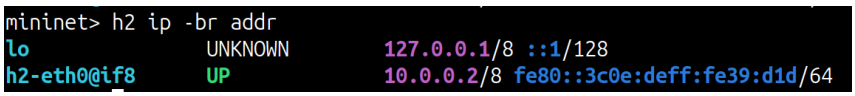


FIGURE 29 – Adresse IP de `h2` : `10.0.0.2/8`

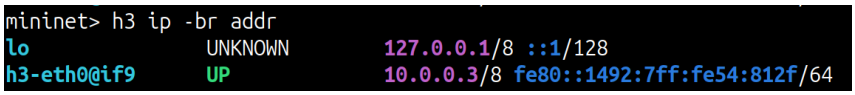


FIGURE 30 – Adresse IP de `h3` : `10.0.0.3/8`

Observation. La topologie est correctement créée avec trois hôtes (`h1`, `h2`, `h3`) reliés au switch `s1`. Chaque hôte possède une adresse dans le réseau `10.0.0.0/8`. Le switch est configuré pour se connecter au contrôleur ONOS sur `tcp:127.0.0.1:6653`.

6.4 Diagnostic et activation du provider OpenFlow

Après le lancement de Mininet, la console ONOS indique `devices` vide. L'inspection d'OVS révèle que `is_connected: false` et l'état `BACKOFF`, signifiant que les tentatives de connexion à ONOS échouent. La cause racine est l'absence du provider OpenFlow dans ONOS.

Listing 18 – Console ONOS : vérification des apps et activation d'OpenFlow

```
onos@root> apps -a -s
onos@root> app activate org.onosproject.openflow
onos@root> apps -a -s
onos@root> devices
```

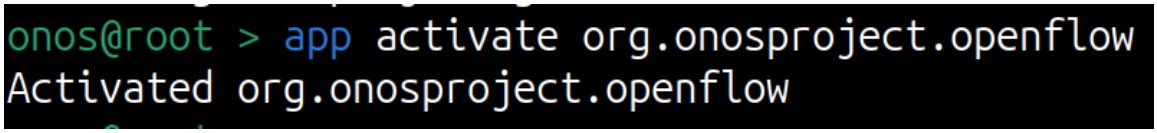


FIGURE 31 – Activation de l'application `org.onosproject.openflow` dans la console ONOS

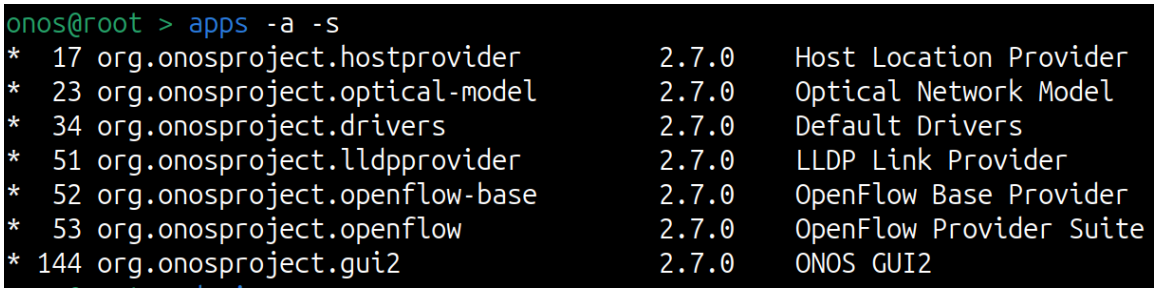


FIGURE 32 – Applications actives après activation : `openflow` apparaît dans la liste

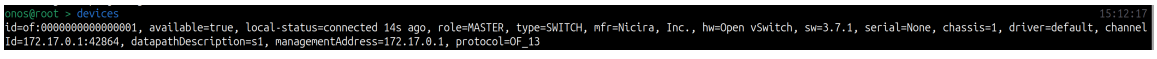


FIGURE 33 – Sortie de `devices` : le switch `s1` apparaît désormais dans ONOS

Explication. Dans ONOS 2.7.0, le provider OpenFlow n'est pas activé par défaut. Sans lui, le port 6653 reste fermé côté ONOS (bien que Docker expose le port, aucun processus OSGi ne l'écoute) et OVS ne peut pas établir la session OpenFlow. L'activation de `org.onosproject.openflow` ouvre le listener et `s1` se connecte immédiatement.

6.5 Vérification de la connexion OVS–ONOS

Listing 19 – Vérification côté OVS : contrôleur et état

```
sudo ovs-vsctl get-controller s1
sudo ovs-vsctl list Controller
sudo ovs-ofctl -O OpenFlow13 show s1
```



FIGURE 34 – Sortie de `ovs-vsctl get-controller s1` : contrôleur configuré sur `tcp:127.0.0.1:6653`

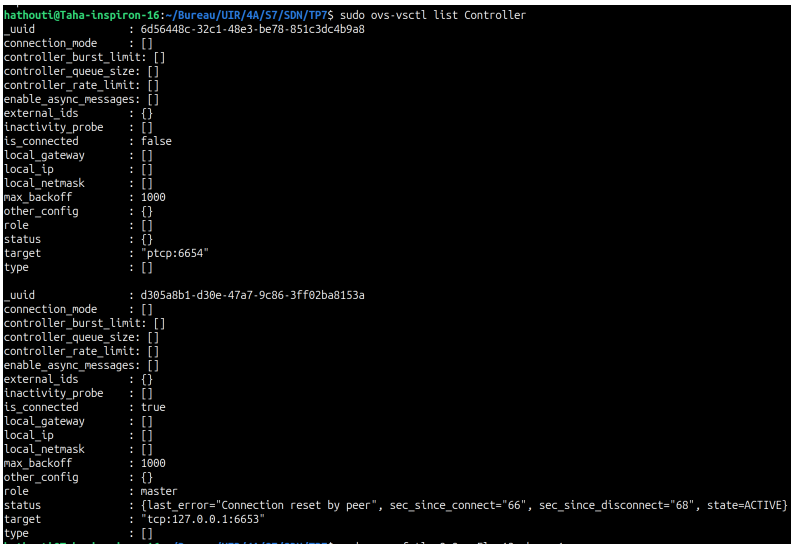


FIGURE 35 – Sortie de `ovs-vsctl list Controller` : `is_connected: true`

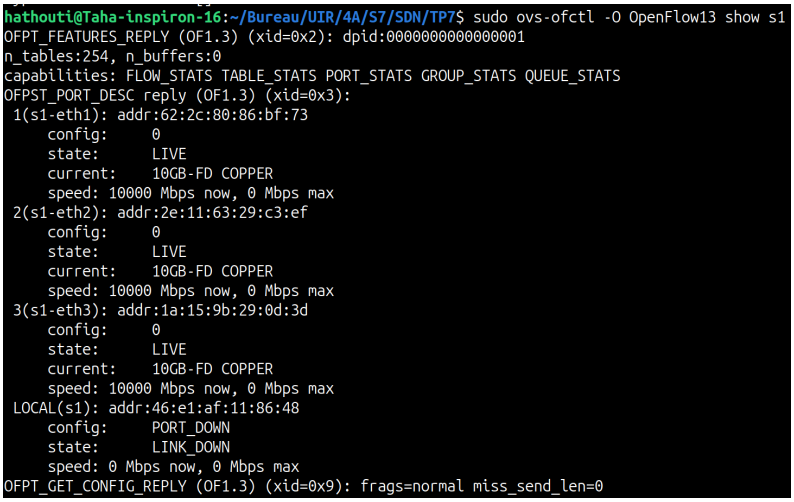


FIGURE 36 – Sortie de `ovs-ofctl -O OpenFlow13 show s1` : switch connecté, ports visibles

Observation. Après activation du provider OpenFlow, `is_connected: true` confirme que `s1` est bien relié à ONOS. La commande `show s1` liste les trois ports (`s1-eth1`, `s1-eth2`, `s1-eth3`) à l'état `LIVE` en OpenFlow 1.3.

7 Étape G — Forwarding réactif et vérification de la connectivité

Objectif : activer l'application `fwd` (Reactive Forwarding) d'ONOS pour permettre la communication entre les hôtes, puis vérifier la connectivité et observer les flux installés automatiquement.

7.1 Test sans forwarding : `pingall` en échec

Listing 20 – Test de connectivité sans l'application fwd

```
mininet> pingall
```

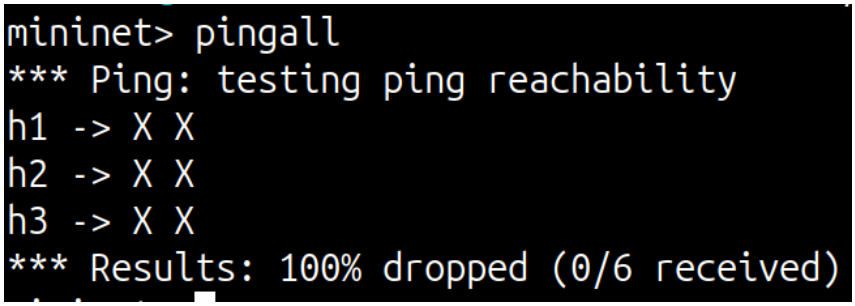


FIGURE 37 – `pingall` en échec : aucune règle de forwarding installée

Observation. Sans application de forwarding dans ONOS, le switch ne sait pas comment transmettre les paquets et les abandonne. Le `pingall` échoue à 100 %.

7.2 Activation de l'application `fwd`

Listing 21 – Activation de l'application Reactive Forwarding

```
onos@root> app activate org.onosproject.fwd
onos@root> apps -a -s
```

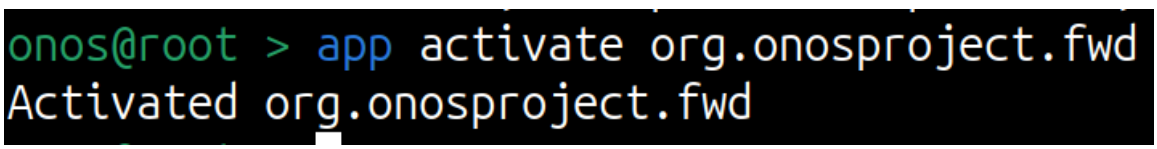


FIGURE 38 – Activation de l'application `org.onosproject.fwd` dans ONOS

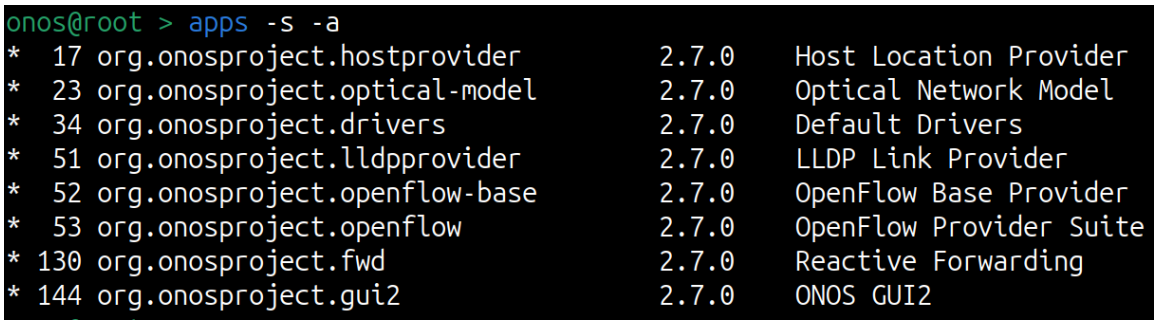


FIGURE 39 – Applications actives après activation : `fwd` apparaît dans la liste

Observation. L'application `org.onosproject.fwd` est maintenant active. Le Reactive Forwarding d'ONOS installe dynamiquement des règles OpenFlow dans le switch au fur et à mesure que le trafic est observé.

7.3 Vérification de la connectivité

Listing 22 – Tests de connectivité après activation de fwd

```
1 mininet> pingall
2 mininet> h1 ping -c 3 h3
```

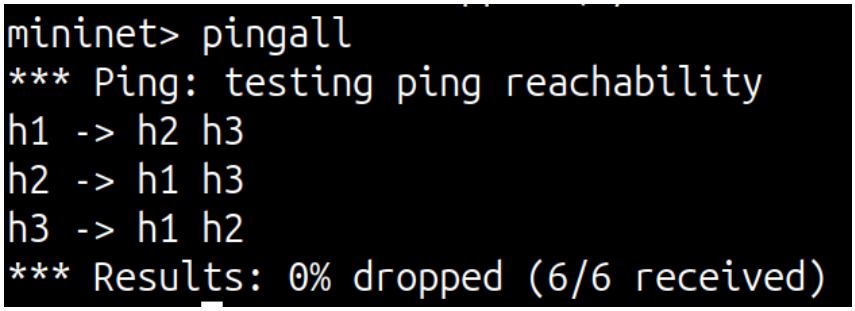


FIGURE 40 – `pingall` réussi : tous les hôtes se joignent (0 % de perte)

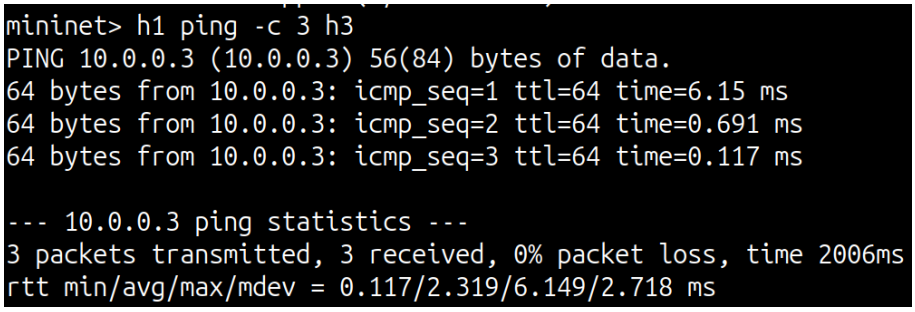


FIGURE 41 – Ping `h1` → `h3` réussi après activation du forwarding réactif

Observation. Le `pingall` réussit avec 0 % de perte. Les six paires de communication (h1↔h2, h1↔h3, h2↔h3) sont opérationnelles. ONOS a appris les hôtes et installé les règles de forwarding dans `s1`.

7.4 Flux OpenFlow installés et hôtes découverts

Listing 23 – Vérification des flux installés et des hôtes découverts

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
onos@root> hosts
onos@root> flows
```

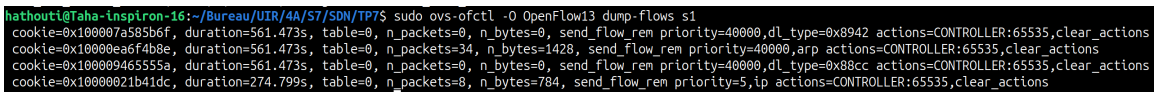


FIGURE 42 – Flux OpenFlow installés dans `s1` par ONOS après `pingall`

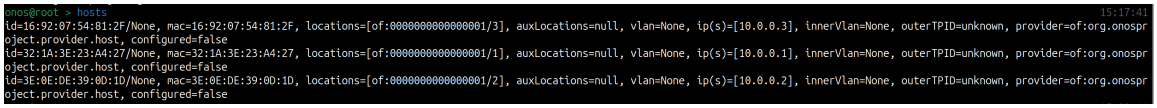


FIGURE 43 – Console ONOS : `hosts` liste les trois hôtes découverts avec leurs adresses MAC et IP

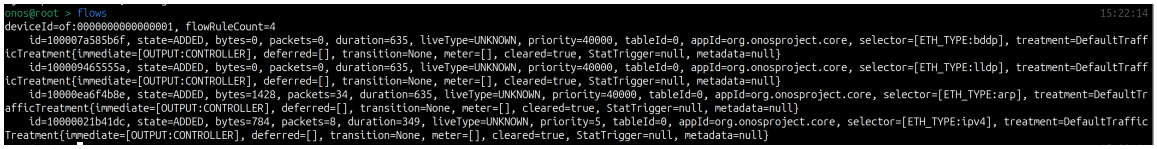


FIGURE 44 – Console ONOS : `flows` liste les règles OpenFlow installées par `fwd`

Observation. Les flux installés par ONOS dans `s1` correspondent aux chemins unicast entre les hôtes (règles sur `eth_dst` avec action `output:N`). La commande `hosts` liste `h1`, `h2` et `h3` avec leurs adresses MAC et IP respectives, confirmant que le Reactive Forwarding a bien découvert les hôtes lors du premier `pingall`. La commande `flows` dans ONOS affiche les mêmes règles avec leur état (`ADDED`).

Hôte	Adresse IP	Port `s1`
`h1`	`10.0.0.1`	port 1
`h2`	`10.0.0.2`	port 2
`h3`	`10.0.0.3`	port 3

TABLE 2 – Mapping hôtes / ports OpenFlow de `s1`

8 Étape H — Interrogation de l'API REST d'ONOS

Objectif : interroger l'API REST d'ONOS via `curl` pour récupérer la liste des équipements gérés, et vérifier la concordance avec ce qu'affiche la console Karaf.

Listing 24 – Requête REST : liste des équipements

```
curl -u onos:rocks http://localhost:8181/onos/v1/devices \
  | python3 -m json.tool
```



FIGURE 45 – Réponse JSON de l'API REST `/onos/v1/devices` : le switch `s1` est listé

Observation. L'API REST renvoie un objet JSON contenant la clé `devices` avec un tableau décrivant le switch `s1` (identifiant ONOS du type `of:0000...`, fabricant `Nicira, Inc.`, version du logiciel OVS, protocole `OF_13`). Cette réponse confirme que l'API REST d'ONOS est opérationnelle et cohérente avec la vue de la console Karaf. Elle peut être exploitée programmatiquement pour automatiser la gestion du réseau SDN.

Champ JSON	Valeur
`type`	`SWITCH`
`available`	`true`
`protocol`	`OF_13`
`manufacturer`	`Nicira, Inc.`
`chassisId`	identifiant unique du switch

TABLE 3 – Principaux champs JSON retournés par `/onos/v1/devices`

9 Arrêt propre de l'environnement

Objectif : quitter Mininet proprement, nettoyer les artefacts réseaux résiduels, puis arrêter le conteneur ONOS.

Listing 25 – Arrêt propre : exit Mininet, nettoyage, arrêt ONOS

```
mininet> exit
sudo mn -c
sudo docker stop onos
```

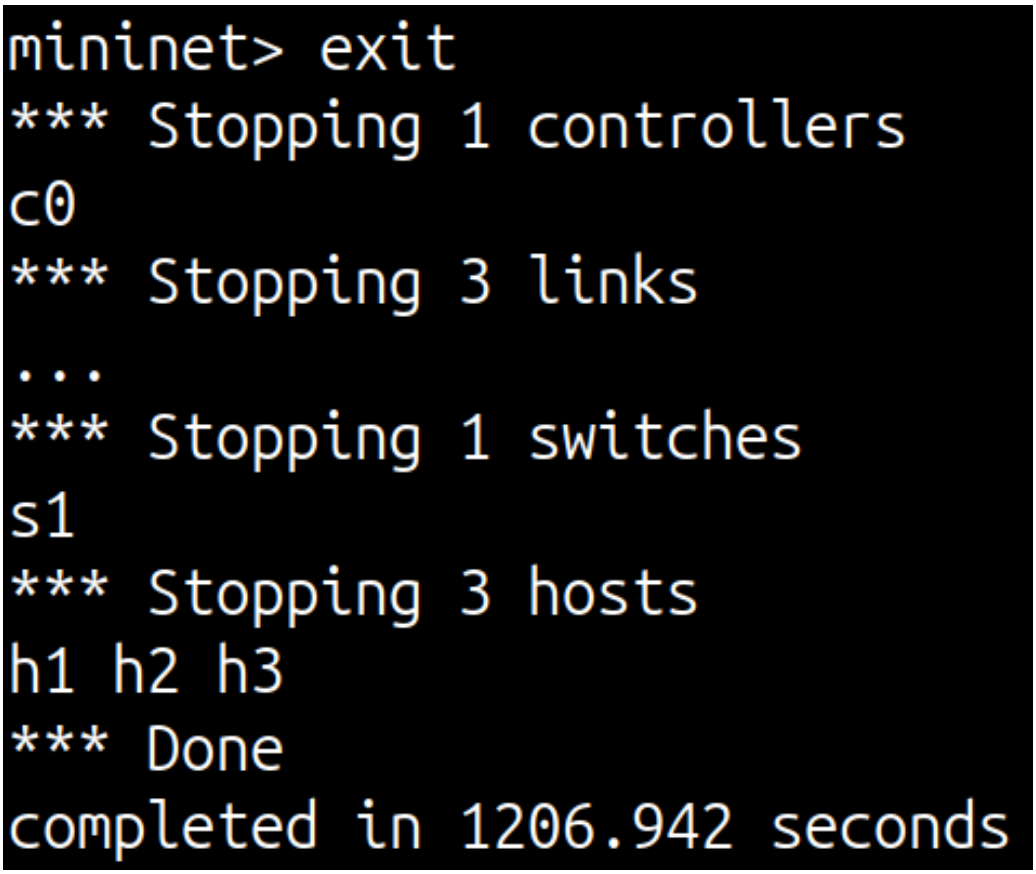


FIGURE 46 – Sortie propre de Mininet avec `exit`



FIGURE 47 – Nettoyage de l'environnement Mininet avec `sudo mn -c`

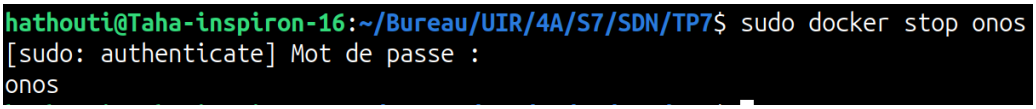


FIGURE 48 – Arrêt du conteneur ONOS avec `docker stop onos`

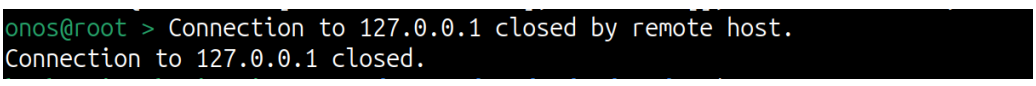


FIGURE 49 – Fermeture de la session SSH Karaf après l'arrêt d'ONOS

Observation. La séquence d'arrêt se déroule sans erreur : `exit` quitte le CLI Mininet, `mn -c` supprime les bridges et namespaces Linux résiduels, `docker stop onos` envoie SIGTERM au conteneur qui s'arrête proprement, et la session SSH Karaf affiche `Connection closed by remote host` confirmant l'extinction d'ONOS.

10 Questions de compréhension et livrables

Question 1 : Pourquoi utilise-t-on Docker pour ONOS, alors que Mininet et OVS restent sur Ubuntu ?

ONOS est une application Java complexe (plateforme OSGi/Karaf, dizaines de bundles, dépendances JVM précises) dont les versions de dépendances peuvent entrer en conflit avec le système hôte. Docker isole l'environnement d'exécution d'ONOS dans un conteneur autonome : sa version de Java, ses bibliothèques et sa configuration sont figées dans l'image `onosproject/onos:2.7.0`, indépendamment de ce qui est installé sur Ubuntu. L'accès à ONOS depuis l'extérieur se fait via les ports exposés (`-p`), ce qui suffit pour qu'OVS et les outils de l'hôte communiquent avec lui.

Mininet et OVS, en revanche, opèrent directement sur le noyau Linux (namespaces réseau, `netfilter`, interfaces virtuelles) et ne peuvent pas être conteneurisés facilement sans accès privilégié au noyau de l'hôte : ils *sont* le plan de données du réseau émulé. Les faire tourner nativement sur Ubuntu évite des problèmes de compatibilité avec les fonctionnalités noyau requises (`openvswitch` module, `network_ns`, etc.).

Question 2 : Quelle est la différence entre les ports TCP 6653, 8181 et 8101 ?

Port	Protocole / Service	Usage
`6653`	OpenFlow 1.3 (TCP)	Canal de contrôle entre OVS (switch) et ONOS (contrôleur). C'est sur ce port qu'OVS envoie les messages `PACKET_IN` et reçoit les messages `FLOW_MOD` / `PACKET_OUT`. Port standard OpenFlow depuis la version 1.4 (anciennement 6633).
`8181`	HTTP / REST API	Interface Web ONOS (GUI2) et API REST Northbound (`/onos/v1/...`). Permet d'interroger ou de configurer ONOS depuis un navigateur ou via `curl`.
`8101`	SSH / Apache Karaf	Console interactive Karaf. Donne accès aux commandes ONOS (`apps`, `devices`, `flows`, `hosts`, etc.) via un shell SSH.

Table 4 – Rôle des trois ports exposés par le conteneur ONOS

Question 3 : Un switch apparaît dans `devices`, mais `pingall` échoue. Que peut-on en déduire ?

La présence du switch dans `devices` signifie uniquement que le **canal de contrôle OpenFlow** est établi : OVS et ONOS ont échangé les messages `HELLO` et `FEATURES_REQUEST/REPLY`, et ONOS gère désormais le switch. Mais cela n'implique pas que des **règles de forwarding** soient installées dans la table de flux.

Si `pingall` échoue dans cet état, on peut déduire qu'aucune application SDN active n'installe de règles dans le switch : sans l'application `org.onosproject.fwd` (ou une application équivalente), chaque paquet reçu par `s1` génère un `PACKET_IN` vers ONOS, qui l'ignore faute de logique de décision, ou le switch l'abandonne selon son `fail-mode`. La connexion contrôleur/switch est une condition nécessaire mais pas suffisante à la connectivité entre les hôtes.

Question 4 : Quelle commande permet de vérifier que `s1` est effectivement connecté à ONOS ?

Deux commandes complémentaires permettent de le vérifier :

Listing 26 – Vérification de la connexion OVS–ONOS

```
# C t OVS : tat de la session OpenFlow
sudo ovs-vsctl list Controller

# C t ONOS : liste des switches g r s
onos@root> devices
```

Côté OVS, le champ `is_connected: true` dans la sortie de `ovs-vsctl list Controller` confirme que la session OpenFlow est active. Le champ `state` passe de `BACKOFF` (tentative de reconnexion) à `ACTIVE` une fois la connexion établie.

Côté ONOS, la commande `devices` affiche le switch avec son identifiant OpenFlow (`of:0000...`) et son état `available=true`. Ces deux vérifications croisées permettent de confirmer que la connexion est bidirectionnelle.

Question 5 : Que change l'activation de `org.onosproject.fwd` ? Distinguer `PACKET_IN` et `FLOW_MOD`.

L'application `org.onosproject.fwd` (Reactive Forwarding) implémente un algorithme de *MAC learning* au niveau du contrôleur. Son activation change le comportement du plan de données comme suit :

Avant activation : lorsque `s1` reçoit un paquet sans règle correspondante, il génère un `PACKET_IN` vers ONOS. Sans application active, ONOS ne répond pas avec de règles : le paquet est perdu ou retourné non traité.

Après activation : `fwd` réceptionne le `PACKET_IN`, extrait l'adresse MAC source et le port d'entrée, les enregistre dans le *Host Store*, puis :

— si la destination est connue : envoie un `FLOW_MOD` pour installer une règle de forwarding dans `s1` (les paquets suivants du même flux sont traités directement par le switch), et un `PACKET_OUT` pour retransmettre le paquet courant ;

— si la destination est inconnue : envoie un `PACKET_OUT` en broadcast (flooding) sur tous les ports sauf le port source.

PACKET_IN	FLOW_MOD
Message envoyé par le switch **vers** le contrôleur quand un paquet ne correspond à aucune règle existante. Il contient le contenu du paquet (ou un extrait) et les métadonnées (port d'entrée, identifiant du switch).	Message envoyé par le contrôleur **vers** le switch pour ajouter, modifier ou supprimer une règle dans la table de flux. Une règle type : *si eth_dst = AA :BB :CC :DD :EE :FF, alors output :2.*
Déclenché par l'absence de règle (table-miss)	Installe la règle pour éviter les futurs PACKET_IN pour ce flux

TABLE 5 – Distinction entre PACKET_IN (switch → contrôleur) et FLOW_MOD (contrôleur → switch)

Question 6 : Quelle différence entre `flows` dans ONOS et `dump-flows` dans OVS ?

	`flows` **(console ONOS)**	`ovs-ofctl dump-flows s1`
Source	Vue *logique* côté contrôleur : règles que ONOS a décidé d'installer et qu'il croit présentes dans le switch	Vue *physique* côté dataplane : règles réellement présentes dans la table de flux du switch au moment de la requête
Contenu	Objets FlowRule ONOS avec priorité, sélecteur, traitement, état (`ADDED`, `PENDING_ADD`), cookies	Entrées OpenFlow brutes avec compteurs de paquets/octets, timeouts, actions
Utilité	Vérifier ce que ONOS pense avoir installé, identifier les règles en attente ou rejetées	Vérifier ce qui est réellement programmé dans le switch, confirmer la cohérence avec la vue ONOS
Discordance possible	Si `flows` montre `ADDED` mais `dump-flows` ne montre pas la règle → échec de programmation OpenFlow ; si `dump-flows` montre une règle absente de `flows` → règle installée manuellement (`ovs-ofctl add-flow`) sans passer par ONOS	

TABLE 6 – Comparaison `flows` ONOS vs `dump-flows` OVS

Conclusion

Ce TP a permis de mettre en place un environnement SDN complet en déployant le contrôleur ONOS 2.7.0 dans un conteneur Docker et en le connectant à un switch Open vSwitch via le protocole OpenFlow 1.3. Les étapes principales réalisées sont :

1. **Vérification des prérequis** : système Ubuntu 64 bits, RAM suffisante, Docker opérationnel et Mininet/OVS installés.
2. **Déploiement d'ONOS** : pull de l'image `onosproject/onos:2.7.0`, lancement avec exposition des ports 6653, 8181 et 8101.
3. **Création de la topologie** : réseau `single,3` dans Mininet, switch OVS en mode OpenFlow 1.3 pointant vers `tcp:127.0.0.1:6653`.
4. **Diagnostic et résolution** : activation manuelle de `org.onosproject.openflow` nécessaire car le provider n'est pas activé par défaut dans ONOS 2.7.0.
5. **Forwarding réactif** : activation de `org.onosproject.fwd`, `pingall` réussi à 100 %, flux OpenFlow installés dynamiquement par ONOS.
6. **API REST** : vérification programmatique de l'état du réseau via `/onos/v1/devices`.

Leçon clé. Dans un environnement SDN avec ONOS, la séparation entre le *transport OpenFlow* (port réseau) et le *provider applicatif* (application OSGi) est fondamentale : un port ouvert par Docker ne suffit pas, l'application ONOS qui l'écoute doit être explicitement activée. Le Reactive Forwarding démontre comment un contrôleur SDN peut automatiser la programmation du plan de données sans intervention manuelle, contrairement à l'approche manuelle par `ovs-ofctl` du TP6.